3.1.1 El Objeto Global `Temporal`

El objeto **`Temporal`** es la API moderna de JavaScript diseñada para **reemplazar por completo al antiguo y problemático objeto `Date`**. Forma parte de las especificaciones de **ES2026**.

Resuelve de raíz los dolores de cabeza históricos de los desarrolladores: los meses ahora empiezan en 1 (Enero es 1, no 0), los objetos son **inmutables** (operaciones como añadir días devuelven una nueva instancia sin modificar la original) y ofrece un soporte nativo excelente para zonas horarias y operaciones matemáticas con fechas.

Soporte actual en el Navegador (2026)

El soporte nativo en los motores de los navegadores se encuentra disponible en **Google Chrome / Edge, Mozilla Firefox** y no soportado en **Safari (**solo mediante *Technical Preview)*.

Para entornos de producción globales: Dado que siempre hay usuarios con navegadores ligeramente desactualizados, se sigue recomendando el uso de un **polyfill** como el oficial `@js-temporal/polyfill` o alternativas ligeras como `temporal-polyfill`.

Ejemplo: Crear un elemento `<script>` dinámicamente mediante JavaScript para descargar el archivo de la CDN solo si el navegador carece de la API nativa.

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>Bienvenidos!</title>
    <link rel="stylesheet"
href="https://unpkg.com/@picocss/pico@latest/css/pico.min.css">
  </head>
  <body>
      <h1 class="title">Carga Condicional de Temporal</h1>

    <script>
        // Función para cargar el script de forma asíncrona
        function cargarScript(src) {
            return new Promise((resolve, reject) => {
                const script = document.createElement('script');
                script.src = src;
                script.onload = resolve;
                script.onerror = reject;
                document.head.appendChild(script);
            });
        }

        async function comprobarYArrancar() {
            // Comprobación de disponibilidad nativa
            if (!("Temporal" in window)) {
```

```
                console.log("Cargando polyfill desde CDN...");
                // Cambia la URL por la de la versión exacta que
uses de @js-temporal/polyfill
                await
cargarScript('https://cdn.jsdelivr.net/npm/temporal-polyfill@1.0.8
/full/global.min.js');
            } else {
                console.log("Temporal nativo detectado.");
            }

            // Código principal
            const ahora = Temporal.Now.instant();
            console.log("Momento exacto:", ahora.toString());
        }
        //delete window.Temporal;
        comprobarYArrancar();
    </script>
</body>
</html>
```

¿Por qué es la mejor estrategia?

- **Ahorro de ancho de banda:** Los usuarios con navegadores modernos no descargarán los kilobytes que pesa el polyfill, haciendo que tu web cargue notablemente más rápido.
- **Rendimiento óptimo:** Siempre que se pueda se ejecutará el código compilado nativo en C++/Rust del motor del navegador, el cual es infinitamente más rápido que cualquier polyfill de JavaScript.

Tipos de objetos principales

A diferencia de `Date`, que intentaba controlarlo todo en un único objeto, `Temporal` se divide en "varios" según lo que necesites representar:

1. `Temporal.PlainDate` (Fechas sin hora ni zona horaria)

Ideal para cumpleaños, días festivos o eventos del calendario escolar donde la hora local no importa.

```
// Crear una fecha: Año, Mes, Día (Enero = 1)

const cumpleanos = new Temporal.PlainDate(2026, 11, 29);

console.log(cumpleanos.toString()); // "2026-11-29"

// Sumar tiempo de manera inmutable
```

```
const proximaSemana = cumpleanos.add({ days: 7 }); // "2026-11-29"


const navidad = Temporal.PlainDate.from("2026-12-25");

console.log(navidad.year);        // 2026
console.log(navidad.month);       // 12 (Diciembre)
console.log(navidad.day);         // 25
console.log(navidad.dayOfWeek);   // 5 (Viernes)
console.log(navidad.daysInMonth); // 31
console.log(navidad.inLeapYear);  // false (2026 no es bisiesto)



// Método estático .compare() para ordenar fechas
const fechaA = Temporal.PlainDate.from("2026-05-10");
const fechaB = Temporal.PlainDate.from("2026-08-20");
console.log(Temporal.PlainDate.compare(fechaA, fechaB)); // -1 (A
es antes que B)

// Método .add() y .subtract() (Inmutables)
const añoNuevo7d = fecha1.add({ days: 7 });
const añoNuevo7m = añoNuevo7d.add({ months: 7 });
const añoNuevo7y = añoNuevo7m.add({ years: 7 });
console.log(añoNuevo7y.toString());
const añoNuevo7ys = añoNuevo7y.subtract({ years: 7 });
const añoNuevo7ms = añoNuevo7ys.subtract({ months: 7 });
const añoNuevo7ds = añoNuevo7ms.subtract({ days: 7 });
console.log(añoNuevo7ds.toString()); // "2027-01-01"


// Método .with() (Modificar una propiedad de forma segura)
const navidadSiguienteAño = navidad.with({ year: 2027 });
console.log(navidadSiguienteAño.toString()); // "2027-12-25"

// Método .equals()
console.log(navidad.equals(navidadSiguienteAño)); // false
```

```
// Métodos de formato
console.log(navidad.toLocaleString("es-ES", { weekday: "long" }));
// "viernes"
```

2. `Temporal.PlainTime` (Horas sin fecha)

Ideal para alarmas o el horario de apertura de un comercio ("abrimos a las 09:00", sin importar el día o el país).

```
const horaApertura = new Temporal.PlainTime(9, 30, 0); // 09:30 AM

const horaReunion = Temporal.PlainTime.from("14:30:15.500");

console.log(horaReunion.hour);        // 14
console.log(horaReunion.minute);      // 30
console.log(horaReunion.second);      // 15
console.log(horaReunion.millisecond); // 500

// Método .add() y .subtract()
const finReunion = horaReunion.add({ hours: 1, minutes: 15 });
console.log(finReunion.toString()); // "15:45:15.5"

// Método .with()
const horaEnPunto = horaReunion.with({ minute: 0, second: 0,
millisecond: 0 });
console.log(horaEnPunto.toString()); // "14:00:00"
```

3. `Temporal.ZonedDateTime` (Fecha + Hora + Zona Horaria)

Imprescindible para citas internacionales, vuelos o eventos que ocurren en un momento exacto del planeta en una ubicación específica.

```
const citaZonificada = Temporal.ZonedDateTime.from({
  timeZone: 'Europe/Madrid',
  year: 2026,
  month: 12,
  day: 25,
  hour: 17,
```

```
  minute: 0
});

const eventoParis =
Temporal.ZonedDateTime.from("2026-11-08T15:00:00+01:00[Europe/Paris]");


console.log(eventoParis.timeZoneId); // "Europe/Paris"
console.log(eventoParis.hour);       // 15
console.log(eventoParis.offset);     // "+01:00"

// Método .withTimeZone() (Saber qué hora es en otro lugar del mundo en ese mismo instante)
const horaEnTokyo = eventoParis.withTimeZone("Asia/Tokyo");
console.log(horaEnTokyo.toString()); //
"2026-11-09T23:00:00+09:00[Asia/Tokyo]"
```

4. `Temporal.Duration` (Lapsos de tiempo)

El tipo de dato que `Date` nunca tuvo. Representa una cantidad de tiempo transcurrido (por ejemplo, "2 horas y 15 minutos").

```
const fecha1 = Temporal.PlainDate.from('2026-01-01');
const fecha2 = Temporal.PlainDate.from('2026-03-15');

// Calcular la distancia exacta entre dos fechas sin hacer matemáticas de milisegundos

const diferencia = fecha1.until(fecha2);

console.log(diferencia.toString()); // Devuelve la duración en meses y días exactos 'P73D' (Periodo de 73 días)



const miViaje = Temporal.Duration.from({ months: 2, days: 5, hours: 12 });


console.log(miViaje.months); // 2
console.log(miViaje.days);   // 5
console.log(miViaje.hours);  // 12
console.log(miViaje.sign);   // 1 (indica que es una duración positiva)
```

```
const hoy = Temporal.PlainDate.from("2026-11-08");
const vacaciones = Temporal.PlainDate.from("2026-12-20");

// Método .until() (Cuánto falta DESDE hoy HASTA las vacaciones)
const tiempoRestante = hoy.until(vacaciones);
console.log(tiempoRestante.toString()); // "P42D" (Periodo de 42 días)

// Método .since() (Cuánto tiempo ha pasado DESDE las vacaciones HASTA hoy - Da negativo)
const tiempoTranscurrido = hoy.since(vacaciones);

console.log(tiempoTranscurrido.sign); // -1


const ahora = Temporal.Now.zonedDateTimeISO();
const evento = Temporal.ZonedDateTime.from('2026-11-09T20:00:00[Europe/Madrid]');

const ahora = Temporal.Now.zonedDateTimeISO();
const evento = Temporal.ZonedDateTime.from('2026-11-19T20:00:00[Europe/Madrid]');

// Le pedimos explícitamente la diferencia desglosada
const duracion = ahora.until(evento, { largestUnit: 'hours' });
console.log(`Faltan: ${duracion.hours} horas, ${duracion.minutes} minutos, ${duracion.seconds} segundos,  ${duracion.milliseconds} milisegundos, ${duracion.microseconds} microsegundos.`);
const totalSegundos = duracion.total({ unit: 'second' });
console.log(totalSegundos);
```

5. `Temporal.Now`

No genera instancias directas; es un espacio de nombres estático para capturar el tiempo real del dispositivo.

```
// Métodos de captura del presente
const timestampUTC = Temporal.Now.instant();
console.log(timestampUTC.toString()); // Ejemplo: "2026-11-08T22:31:00Z"

const fechaLocal = Temporal.Now.plainDateISO();
console.log(fechaLocal.toString()); // "2026-11-08"
```

```
const horaLocal = Temporal.Now.plainTimeISO();
console.log(horaLocal.toString()); // "23:31:00"

const zonaHorariaUsuario = Temporal.Now.timeZoneId();
console.log(zonaHorariaUsuario); // "Europe/Madrid"
```

Gestión Automática del Horario de Verano (DST)

Con el objeto `Date` tradicional, si sumabas 24 horas a una fecha justo el día del cambio de hora, tu reloj se desajustaba. `Temporal` sabe exactamente cuándo cambia la hora en cada rincón del planeta.

Ejemplo: El cambio al horario de verano en Madrid (Marzo)

En España, el último domingo de marzo a las **02:00 AM** el reloj salta directamente a las **03:00 AM** (esa hora no existe en el calendario).

```
// Creamos una cita justo un minuto antes del cambio de hora
const citaPreCambio = Temporal.ZonedDateTime.from({
  timeZone: 'Europe/Madrid',
  year: 2026,
  month: 3,
  day: 29,
  hour: 1,
  minute: 59,
  second: 0
});

console.log("Antes del cambio:", citaPreCambio.toString());
// "2026-03-29T01:59:00+01:00[Europe/Madrid]"

// Sumamos solo 2 minutos a la cita
const citaPostCambio = citaPreCambio.add({ minutes: 2 });

// El reloj saltó automáticamente a las 03:01 y el desfase
(offset) cambió a +02:00
console.log("Después del cambio:", citaPostCambio.toString());

// "2026-03-29T03:01:00+02:00[Europe/Madrid]"
```

Cuando los países cambian al horario de invierno, el reloj se atrasa una hora (por ejemplo, a las 03:00 AM vuelve a ser la 02:00 AM). Eso significa que **esa hora se repite dos veces en el mismo día**. ¿Cómo sabe JavaScript a cuál te refieres si creas una cita en ese momento exacto?

`Temporal` te permite pasar una propiedad de desambiguación (`disambiguation`) para decidir el comportamiento de forma explícita:

```
// Intentamos crear una fecha en una hora duplicada por el cambio de invierno
const horaConflictiva = Temporal.ZonedDateTime.from({
  timeZone: 'Europe/Madrid',
  year: 2026,
  month: 10,
  day: 25,
  hour: 2,
  minute: 30
}, { disambiguation: 'earlier' }); // Opciones: 'earlier' (primera vez), 'later' (segunda vez), 'reject' (lanza error)

console.log(horaConflictiva.toString());
```

Comparar y ordenar una lista de fechas

- **Antes (`Date`):** `lista.sort((a, b) => a.getTime() - b.getTime())`

Ahora (`Temporal`):

```
const fechas = [
  Temporal.PlainDate.from("2026-12-25"),
  Temporal.PlainDate.from("2026-01-01"),
  Temporal.PlainDate.from("2026-06-15")
];

// El método estático .compare devuelve -1, 0 o 1 de forma perfecta para ordenamientos
fechas.sort(Temporal.PlainDate.compare);
console.log(fechas.map(f => f.toString()));
     // ["2026-01-01", "2026-06-15", "2026-12-25"]
```

Clases de Combinación Parcial (`PlainMonthDay` y `PlainYearMonth`)

A veces no necesitas una fecha completa (Año-Mes-Día). Guardar esos datos a medias en el objeto `Date` antiguo obligaba a inventarse valores falsos (como poner año `1900` para un cumpleaños). `Temporal` introduce tipos de datos nativos para esto:

`Temporal.PlainMonthDay`: Ideal para eventos recurrentes anuales como **cumpleaños o aniversarios** (solo importa el día y el mes).

```
const cumple = Temporal.PlainMonthDay.from({ month: 10, day: 24
});

// Puedes averiguar qué día de la semana caerá en un año concreto
const en2027 = cumple.toPlainDate({ year: 2027 });
console.log(en2027.dayOfWeek); // 7 (Domingo)
```

`Temporal.PlainYearMonth`: Ideal para **fechas de caducidad de tarjetas de crédito** o periodos de facturación (solo importa el año y el mes).

```
const caducidadTarjeta = Temporal.PlainYearMonth.from("2030-06");
console.log(caducidadTarjeta.daysInMonth); // 30
```

Soporte Nativo para Múltiples Calendarios

El mundo no solo se rige por el calendario occidental (Gregoriano). Si desarrollas aplicaciones internacionales, `Temporal` te permite cambiar el motor de calendario subyacente (como el **Hebreo, Islámico, Chino, Budista o Japonés**) con una sola línea de código, adaptando los meses y días automáticamente sin romper tus operaciones matemáticas:

```
// Creamos una fecha en el calendario gregoriano tradicional
const fechaComun = Temporal.PlainDate.from("2026-11-08");

// La transformamos al calendario islámico
const fechaIslamica = fechaComun.withCalendar("islamic-umalqura");

console.log(fechaIslamica.toString()); // Ejemplo:
"1448-05-28[u-ca=islamic-umalqura]"
console.log(fechaIslamica.month); // Devuelve el mes
correspondiente a su calendario
```

La gran diferencia filosófica: Inmutabilidad estricta

Si hay algo que debes recordar siempre al usar `Temporal`, es que **nunca modifica tus variables**.

```
// CON DATE (El peligro de la mutación)
const fechaDate = new Date("2026-11-08");
fechaDate.setDate(20);
console.log(fechaDate.getDate()); // 20 (La variable original mutó y se destruyó el valor '8')

//  CON TEMPORAL (Diseño funcional seguro)
const fechaTemporal = Temporal.PlainDate.from("2026-11-08");
const nuevaFecha = fechaTemporal.with({ day: 20 });

console.log(fechaTemporal.day); // 8  (Sigue intacta)

console.log(nuevaFecha.day);    // 20 (El resultado está a salvo en la nueva constante)
```

Actividad Práctica 3.1.8: "Gestor de Plazos de Matrícula y Convocatorias con Temporal"

En el departamento del IES Kursaal se necesita un módulo en cliente para gestionar el calendario de matriculación de las pruebas libres y periodos de reclamación. El sistema actual basado en Date sufre errores constantes debido al desfase de zonas horarias y a la mutación accidental de fechas compartidas. Se solicita desarrollar un prototipo utilizando la **Temporal**.

Crea un script que realice las siguientes operaciones:

1. **Definición de fechas base:** Crea la fecha de publicación provisional de notas fijada para el **15 de junio de 2027** mediante Temporal.PlainDate.
2. **Cálculo de plazos legales (Inmutabilidad):**
    - El periodo de alegaciones dura exactamente **3 días hábiles** (asume 3 días naturales para el cálculo). Obtén la fecha de fin de alegaciones sumando ese lapso con .add().
    - La resolución definitiva se publica **2 semanas** después del fin de alegaciones. Calcula dicha fecha.
    - Demuestra por consola que la fecha original de publicación de notas no ha mutado tras realizar las sumas. (fechaPublicacionNotas.toString() === "2027-06-15" es true)
3. **Auditoría de cuenta atrás:**
    - Captura la fecha actual del sistema con Temporal.Now.plainDateISO().
    - Calcula el tiempo restante (en meses y días) desde hoy hasta el inicio de las alegaciones utilizando .until().

4. **Sincronización de apertura de sede internacional:**
    - La plataforma de matriculación abre a las 09:00:00 en la sede de Madrid (Europe/Madrid).
    - Proyecta ese instante exacto para un alumno que se conecta desde Tokio (Asia/Tokyo) utilizando Temporal.ZonedDateTime y su método .withTimeZone().